Two Failures of Self-Consistency in the Multi-Step Reasoning of LLMs

Angelica Chen *ac5968@nyu.edu*
Center for Data Science, New York University

Jason Phang *zp489@nyu.edu*
Center for Data Science, New York University

Alicia Parrish *avp295@nyu.edu*
Department of Linguistics, New York University; Google

Vishakh Padmakumar *vp1271@nyu.edu*
Center for Data Science, New York University

Chen Zhao *cz1285@nyu.edu*
NYU Shanghai; Center for Data Science, New York University

Samuel R. Bowman *sb6065@nyu.edu*
Center for Data Science, New York University; Anthropic

Kyunghyun Cho *kc119@nyu.edu*
Center for Data Science, New York University

Reviewed on OpenReview: *https://openreview.net/forum?id=5nBqY1y96B*

arXiv:2305.14279v4 [cs.CL] 2 Feb 2024

Abstract

Large language models (LLMs) have achieved widespread success on a variety of in-context few-shot tasks, but this success is typically evaluated via correctness rather than consistency. We argue that self-consistency is an important criteria for valid multi-step reasoning in tasks where the solution is composed of the answers to multiple sub-steps. We propose two types of self-consistency that are particularly important for multi-step reasoning – hypothetical consistency (a model's ability to predict what its output would be in a hypothetical other context) and compositional consistency (consistency of a model's final outputs when intermediate sub-steps are replaced with the model's outputs for those steps). We demonstrate that multiple variants of the GPT-3/-4 models exhibit poor consistency rates across both types of consistency on a variety of tasks.

1 Introduction

An important property of logically valid machine learning systems is *self-consistency* – *i.e.*, the requirement that no two statements given by the system are contradictory. Pre-trained large language models (LLMs), despite demonstrating impressive few-shot accuracy on a variety of multi-step reasoning tasks, often give inconsistent responses to questions (Mitchell et al., 2022; Kassner et al., 2021) and factual knowledge-seeking prompts (Elazar et al., 2021). Without self-consistency, it is difficult to consider LLMs reliable or trustworthy systems. Elazar et al. (2021) defines self-consistency as the invariance of an LLM's responses across different types of semantics-preserving *prompt transformations*. In this work, we seek to introduce and explore LLM self-consistency over two new types of transformations (shown in Figure 1) that we argue are important for valid multi-step reasoning.

Hypothetical Transformations A *hypothetical transformation* is an indirect phrasing of a prompt that queries the model for what its response would hypothetically be in some other context, such as "what would your response to `<prompt>` be?" or "what would the next 5 words in your completion of `<prompt>` be?" Consistency over hypothetical transformations implies that an LLM has some stored knowledge or computational path for determining what its response would be to some prompt p without explicitly being prompted with exactly p itself. This can be useful for prompts involving multi-step reasoning, where the LLM must have knowledge of its responses to the earlier steps in order to compute its responses to downstream steps. Like in Figure 1, given the prompt "What is the runtime of the best movie of 2022" the LLM must either have stored or computed its response to "what is the best movie of 2022?" in order to answer the full prompt.

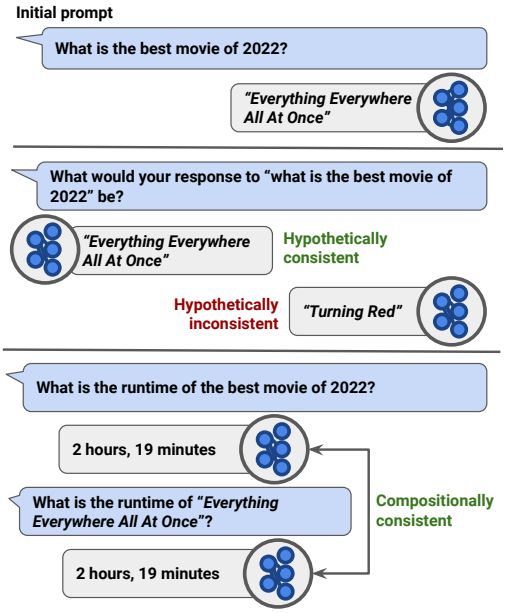

Figure 1: An overview of the two types of self-consistency failures we identify in LLMs.

Compositional Transformations For a prompt that involves multiple interdependent steps of reasoning, a *compositional transformation* consists of replacing some intermediate step with the model's output to the previous step. In the previous example, if the LLM outputs the response "*Everything Everywhere All At Once*" to the prompt "What is the best movie of 2022?," then the prompt "What is the runtime of *Everything Everywhere All At Once*?" is a compositional transformation of "What is the runtime of the best movie of 2022?" (See Figure 1.) Consistency over compositional transformations is also important for logically valid multi-step reasoning when the LLM must give a direct response – without it, the LLM may give contradictory direct responses to different multi-step prompts that are in fact querying for the same thing.

In this work, we investigate the degree to which LLMs are self-consistent on these prompt transformations across a variety of tasks. We formalize our definitions of hypothetical and compositional consistency (Section 2) and show empirically that a wide range of pre-trained language models demonstrate low consistency rates on both hypothetical transformations (Section 3) and compositional transformations (Section 4).

2 Formalizing Consistency

To make more precise our definitions of consistency and the semantics-preserving transformations that they entail, we first formalize our definitions prior to conducting our empirical evaluations.

2.1 Preliminaries

Let vocabulary $\mathcal{V}$ be a finite set of tokens, $\mathcal{V}^*$ be the set of all possible finite-length sequences formed by concatenating zero or more tokens from $\mathcal{V}$, and $p_\theta : \mathcal{V} \to \{0, 1\}$ be an auto-regressive language model that defines a probability distribution over tokens $v \in \mathcal{V}$. p_θ can be used to generate a sequence via greedy decoding as follows:

$$\tilde{y}_t = \arg\max_{v \in \mathcal{V}} \log p_\theta(y_t = v \mid c; \tilde{y}_{<t}) \tag{1}$$

given some context sequence $c \in \mathcal{V}^*$, until some time step T for which $\tilde{y}_T = \texttt{[EOS]}$, the end-of-sequence token. For ease of notation, we denote the greedy decoding output of a model p_θ as

$$g_{p_\theta}(c) = (\arg\max_{v \in \mathcal{V}} p_\theta(y = v|c), \cdots, \arg\max_{v \in \mathcal{V}} p_\theta(y = v|c; \tilde{y}_{<T})). \tag{2}$$

We also define an operator $\sim$ that indicates when two strings are *semantically equivalent*. Although the precise definition of semantic equivalence will vary across different tasks, we use it to loosely refer to pairs of strings that can be used interchangeably (give or take syntactic adjustments) without changing the meaning of the overall utterance. Lastly, the $\sim$ operator is also reflexive, symmetric, and transitive.

2.2 Composing prompts

Reasoning with language often also involves composing prompts – for instance, we might ask "what is the answer to $2 \times 3 + 4$?", which can be seen as the composition of a *prompt template* "what is the answer to $_ + 4$?" with the prompt "2×3", where the "_" symbol in the former string is substituted with the latter string. This corresponds to a multi-step task where the model might first answer the prompt "2×3" (yielding $g_{p_\theta}(\text{“}2 \times 3\text{”})$), substitute $g_{p_\theta}(\text{“}2 \times 3\text{”})$ into the template (yielding the composed prompt "what is the answer to $g_{p_\theta}(\text{“}2 \times 3\text{”}) + 4$?," where the $g_{p_\theta}(\text{“}2 \times 3\text{”})$ is replaced with the actual output string), and then answer the filled-in template.

To denote such prompt templates, we define $\mathcal{P}'$, the set of prompts $p \in \mathcal{V}^*$ that contain exactly one "_" symbol. Additionally, the function $f(p', p) : \mathcal{P}' \times \mathcal{V}^* \to \mathcal{V}^*$ denotes substitution of p for the "_" symbol in p'.[1]

f also has some useful properties that we will use in our later definitions:

- We can trivially represent any prompt $p \in \mathcal{V}^*$ as the substitution of itself into the *identity prompt template* "_" by writing $p = f(\text{“}_\text{”}, p)$.
- $p \sim q$ if and only if $f(p', p) \sim f(p', q)$ for all $p' \in \mathcal{P}'$.

2.3 Definitions

We start out by restating the general definition of self-consistency, as it has been commonly defined in past literature (Elazar et al., 2021; Jang et al., 2022).

Definition 2.1 (Self-consistency)**.** p_θ is *self-consistent* if $p \sim q \to g_{p_\theta}(p) \sim g_{p_\theta}(q)$ for all $p, q \in \mathcal{V}^*$.

In other words, a self-consistent model gives semantically-equivalent responses to semantically equivalent prompts. These semantically equivalent pairs of prompts (p, q in Definition 2.1) can take many forms, including hypothetical and compositional transformations.

Definition 2.2 (Hypothetical Transformation)**.** Let $\mathcal{P}'_I$ denote the set of *hypothetical transformation prompt templates*, which are prompt templates $p' \in \mathcal{P}'$ such that $f(p', p) \sim f(_, p) \forall p \in \mathcal{V}^*$. Then the set of *hypothetical transformations* of prompt p can be denoted as $\mathcal{P}_I(p) := \{f(p', p) \,|\, p' \in \mathcal{P}'_I\}$.

Since $f(_, p) \sim p$, a model that is self-consistent must yield $g_{p_\theta}(f(p', p)) \sim g_{p_\theta}(p)$ for all $p' \in \mathcal{P}'_I$.

Although we defined hypothetical transformations with respect to all prompts $p \in \mathcal{V}^*$, our definition of compositional transformations must be more restricted, since we care only to apply compositional transformations to prompts that implicitly encode a compositional task. That is, we are concerned only with prompts that are already compositions – *i.e.*, prompts of the form $f(p', p)$. Furthermore, given some target model p^* that represents the ground truth or gold distribution, the response to a compositional prompt (as generated by p^*) is semantically equivalent to the prompt itself. That is, $f(p', p) \sim g_{p^*}(f(p', p))$. For example, if $p =$ "2" and $p' =$ "$4 + _$", then $f(p', p) =$ "$4 + 2$" and $g_{p^*}(f(p', p)) = g_{p^*}(\text{“}4 + 2\text{”}) = 6 \sim$ "$4 + 2$" $= f(p', p)$, so $f(p', p)$ is compositional.

Definition 2.3 (Compositional prompt)**.** We define the set of *compositional prompts* as $\mathcal{P}_{\text{Comp}} := \{f(p', p) \,|\, f(p', p) \sim g_{p^*}(f(p', p)), p \in \mathcal{V}^*, p' \in \mathcal{P}'\}$ given gold distribution p^*.

Definition 2.4 (Compositional transformation)**.** For prompt compositions $p \in \mathcal{P}_{\text{Comp}}$ and $f(p', p) \in \mathcal{P}_{\text{Comp}}$ both representing compositional tasks, the *compositional transformation* with respect to model p_θ is $f(p', g_{p_\theta}(p))$.

[1] In practice, substituting p into p' may require minor syntactic adjustments for linguistic acceptability, but we omit these in our notation since the semantics remain the same.